CREACIÓN DE UN MODELO DE CLASIFICACIÓN DE SEÑALES DE EMG

1. Introducción

El objetivo de este proyecto es diseñar un modelo de clasificación que consiga categorizar señales de electromiografía (EMG) en base a una serie de métricas calculadas previamente, en función de si corresponden a un sujeto sano o a uno con secuelas de ictus. Para ello, se ha accedido a una base de datos pública [1], que contenía registros de EMG correspondientes a 22 sujetos, entre sanos y patológicos. Entre las adquisiciones se encontraban registros tanto de movimientos concretos ("targets") como de contracciones voluntarias máximas (MVC), de diferentes músculos involucrados en la movilidad de la extremidad superior. En este caso, se han empleado aquellos relativos a las MVC, por ser más fácilmente reproducibles para la validación del modelo final con sujetos adicionales.

2. Descripción de los datos

La base de datos contenía los registros de EMG de 12 personas sanas y 10 personas con secuelas de ictus. De cada individuo se disponían los registros de hasta nueve movimientos concretos y de las contracciones máximas de ocho músculos del brazo/ tren superior, que son el deltoides anterior, el bíceps, el infraespinoso, el deltoides medio, el pectoral mayor, el deltoides posterior, el tríceps y el trapecio superior. Los datos se encontraban en formato *.csv.* La hoja de cálculo presentaba 11 líneas de metadatos, indicando la frecuencia de muestreo, el filtro usado , las unidades o la ganancia del sistema. Los datos se presentaban mediante una columna que recogía el tiempo (en segundos), pareada con una columna que recogía los valores de amplitud, en voltios, en cada instante de tiempo. Las adquisiciones fueron tomadas con una frecuencia de muestreo de 1952 Hz, y filtradas con un filtro de RMS (*Root Mean Square*) de 52 muestras por ventana.

Una particularidad de estos registros es que, en vez de disponer de la señal de EMG en crudo, la base de datos proporcionaba el registro de amplitud de la señal (su envolvente). Esto dificulta el análisis, ya que librerías de *Python* como *Neurokit2*, que disponen de funciones muy útiles para el análisis de señales biomédicas están diseñadas para trabajar con la señal en crudo. Como se puede apreciar en la *Figura 1*, los registros contienen un reposo inicial, seguido de una contracción máxima que precede a un nuevo estado de reposo, todo ello repetido tres veces.

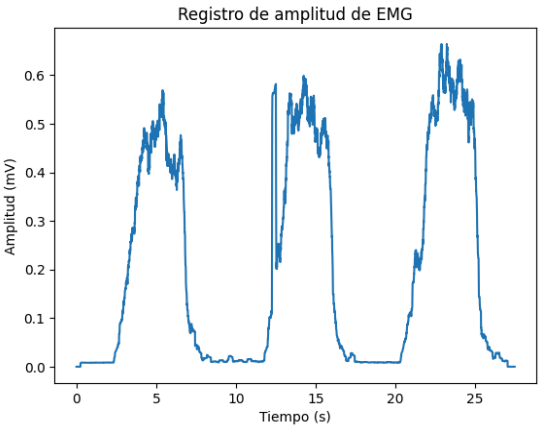


Fig 1. Ejemplo de visualización de un registro de EMG de MVC (bíceps, sujeto sano).

3. Inspección de los datos y "data curation"

Antes de empezar a desarrollar el código se inspeccionaron y visualizaron los datos en busca de errores o posibles valores perdidos. Además, se reorganizaron en diferentes carpetas para facilitar el acceso a los mismos desde el entorno de programación. Para ello, se distribuyeron los datos, nombrados según músculo y número de individuo, en dos carpetas según su estado ("sano", "enfermo"), que a su vez estaban divididas en ocho carpetas diferentes según el músculo al que pertenecía el registro.

```
database/
│
├── sano/
│   ├── biceps/
│   │   ├── MVC biceps_1.csv
│   │   ├── MVC biceps_2.csv
│   │   └── ...
│   ├── deltoides_anterior/
│   │   └── ...
│   └── [resto_de_musculos]/
│
└── enfermo/
    ├── biceps/
    │   ├── MVC biceps_13.csv
    │   └── ...
    ├── deltoides_anterior/
    │   └── ...
    └── [resto_de_musculos]/
```

Para la extracción de características y su posterior registro se creó un primer *script*, *ictus_main*, que recogía las funciones empleadas para la visualización de las señales, la extracción de métricas y su almacenamiento. Se decidió crear un *DataFrame* por cada músculo, registrando los valores obtenidos para cada característica de cada individuo, así como la pertenencia a un grupo u otro mediante una variable binaria ("sano" = 0, "enfermo" = 1).

3.1. Setup e importación de librerías

El entorno elegido para trabajar fue Google Colab, debido a la sencillez de su interfaz y la posibilidad de acceder fácilmente al código almacenado en la nube, así como la no necesidad de tener el lenguaje *Python* instalado en el ordenador. Las primeras líneas fueron destinadas a la instalación e importación de las librerías necesarias:

- matplotlib.pyplot, para la creación de gráficos.
- pandas, para la lectura y manejo de los datos.
- numpy, para operaciones y cálculos.

```
!pip install matplotlib pandas
import matplotlib.pyplot as plt
```

```
import pandas as pd
import numpy as np
```

3.2. Función `plot_emg`

Para visualizar los datos, se creó una función que leyera los datos en formato *.csv*, creando así un DataFrame, para posteriormente extraer las columnas con la información necesaria para representarla adecuadamente. La función recibe como entrada:

- `i`: número del individuo.
- `musculo`: músculo a estudiar.
- `estado`: grupo al que pertenece el individuo.

Estas entradas son acordes a la organización y nomenclatura empleadas en el almacenamiento de los datos, y deben ser adaptadas en caso de cambios, así como la línea de código que accede a la ruta de archivo correspondiente.

Dentro de la función se generan diferentes variables y objetos, empleando funciones nativas de Pandas que permiten obtener y seleccionar los datos:

- `signal`: mediante la funcion de Pandas `read_csv()` se crea un DataFrame con el contenido de la hoja de cálculo. Se añade el argumento `skiprows` para eliminar las líneas con metadatos.
- `amplitud`: del DataFrame anterior se selecciona la columna correspondiente a los valores de amplitud, pasando también las unidades a mV.
- `t`: se toma del DataFrame la columna correspondiente al vector de tiempos.
- Finalmente, se utilizan las funciones de matplotlib.pyplot para dibujar el gráfico, añadir títulos, etc.

```
def plot_emg(i, musculo, estado):

  signal = pd.read_csv(f'/content/database/{estado}/{musculo}/MVC
{musculo}_{i}.csv', skiprows=11)
  amplitud = signal.iloc[:,1]*1e3 #pasar a milivoltios

  t = signal.iloc[:,0]
  plt.plot(t, amplitud)
  plt.xlabel('Tiempo (s)')
  plt.ylabel('Amplitud (mV)')
  plt.title('Registro de amplitud de EMG')

  plt.show()
```

3.3. Función `saca_metricas`

Posteriormente se introdujo una función que, recibiendo los mismo argumentos de entrada que la anterior, cargara el archivo deseado para calcular las métricas correspondientes almacenándolas en una lista. La función carga los datos y selecciona la columna con los valores de amplitud igual que la función anterior. Una vez obtenidos los datos, se calculan diferentes métricas que permitan caracterizar la señal, que en futuras versiones pueden modificarse en busca de un mejor modelo. Son muchas las características que se pueden extraer de una señal como el EMG, para este proyecto inicial se ha decidido escoger las más básicas (máximo, media, mediana) debido a su bajo coste computacional, y alguna un poco más compleja consultada en bibliografía (SSI, RMS):

- **Máximo** (`max`): debido a las características típicas de un paciente que ha padecido un ictus, se esperaría que los valores máximos de amplitud de un individuo sano sean mayores que en un individuo enfermo [2].
- **Media** (`mean`): por la misma razón que la anterior, se esperaría valores inferiores en los individuos enfermos. En individuos con un ictus, con actividad muscular limitada o incluso nula se esperaría que su valor sea más cercano a la línea de base que en individuos sanos [2, 3].
- **Mediana** (`median`): su elección sigue el mismo razonamiento que los anteriores, se incluye por ser más robusta que la media frente a posibles outliers.
- ***Simple Square Integral*** **o Integral Cuadrática Simple** (`ssi`): representa la suma de los valores al cuadrado de la amplitud de la señal a lo largo de un período de tiempo determinado otorgando más peso a las amplitudes más altas. Es decir, mide la energía acumulada que el músculo ha ejercido durante esa contracción. Se esperaría que a mayor reclutamiento muscular, mayores sean los valores de amplitud, por tanto sería de esperar que en individuos enfermos su valor sea significativamente menor [3, 4].
- ***Root Mean Square*** **o Raíz de la Media Cuadrática** (`rms`): representa la potencia de la señal de electromiografía en el dominio del tiempo. La RMS eleva los valores de amplitud al cuadrado antes de promediar., lo que proporciona una mayor sensibilidad física a la potencia efectiva ejercida por el músculo [4].
- **Variable de grupo** (`caso_control`): aunque los datos por sí mismos no indican si el individuo pertenece al grupo de sanos o enfermos, dicha información se encuentra recogida en el nombre del archivo, por lo que la función accede a esta información (incluida previamente como entrada para que `read_csv` pueda acceder a la ruta de archivo correspondiente) y en función de ella la variable `caso_control` toma un valor de 0 o 1.

```
def saca_metricas(i, estado, musculo):
  signal = pd.read_csv(f'/content/database/{estado}/{musculo}/MVC
{musculo}_{i}.csv', skiprows=11)
  amplitud = signal.iloc[:,1]*1e3 #pasar a milivoltios

  # calculo de metricas
  max = amplitud.max()
  mean = amplitud.mean()
```

```
  std = amplitud.std()
  median = amplitud.median()
  ssi = np.sum(amplitud**2)
  rms = np.sqrt(np.mean(amplitud**2))

  if estado == 'enfermo':
    caso_control = 1

  if estado == 'sano':
    caso_control = 0

  return max, mean, std, median, ssi, rms, caso_control
```

3.4. Función `crea_df`

Una vez creada una función capaz de extraer y almacenar las características de la señal de cada individuo, es necesaria un función que barra todos los registros de un mismo músculo, extraiga las características y las incorpore en un mismo DataFrame. En primer lugar se crea una lista vacía (`datos`) donde posteriormente se van a ir almacenando los resultados. Posteriormente, se programa un bucle for por cada grupo, accediendo a cada uno de los registros, llamando a la función anterior para extraer las características y añadiendo el resultado a la matriz de datos. Una vez la matriz está completa, se crea un DataFrame con la función de Pandas correspondiente, indicando el nombre de las columnas, sabiendo en qué orden devuelve `saca_metricas` los resultados. Una vez creado el DataFrame, se desordenan los datos con la función `sample`, que toma una fraccion de los datos (en este caso se coge la totalidad, `frac=1`) y los reindexa (`reset_index(drop=True)`). Esta última acción es necesaria debido a que la efectividad en el aprendizaje de algunos modelos de Machine Learning depende del orden en el que se hayan presentado los datos de entrenamiento, por lo que se evita disponer los casos "sanos" estrictamente al principio del DataFrame, y los "enfermos" al final.

Cabe destacar que el acceso a los datos, de nuevo, va ligado a cómo estos se hayan nombrado y organizado. Además, los bucles están diseñados sabiendo de antemano el número de adquisiciones en cada grupo. Si se reutilizase el código para trabajar con una nueva base de datos sería necesario adaptar y revisar estos parámetros.

```
def crea_df(musculo):

  datos = []

  for i in range(1,13):
    metricas = saca_metricas(i, 'sano', musculo)
    datos.append(metricas)
```

```
  for i in range(1,11):
    metricas = saca_metricas(i, 'enfermo', musculo)
    datos.append(metricas)

  df = pd.DataFrame(datos, columns = ['Max', 'Mean', 'Std',
'Median','SSI', 'RMS', 'Group'])
  df = df.sample(frac=1).reset_index(drop=True)
  df.to_csv(f'{musculo}')

  return df
```

4. Modelo y entrenamiento

En un nuevo *script*, *ictus_training*, se llevó a cabo del proceso de creación, entrenamiento y validación del modelo. En primer lugar se dividió el conjunto de datos entre entrenamiento y validación. Uno de los problemas enfrentados ha sido el pequeño tamaño de la base de datos empleada, ya que utilizar muchos datos para entrenamiento y pocos para validación le resta robustez a este último proceso, al no disponer de una muestra suficientemente representativa con la que probar el modelo. Por otro lado, dejar más datos para validación proporciona unas métricas de rendimiento más fiables, pero supondría una pérdida de calidad de entrenamiento al no disponer de suficientes muestras para ello. Por ello, se decidió emplear un tamaño de 18 individuos en el conjunto de entrenamiento, dejando 4 en el conjunto de validación, buscando un punto intermedio entre ambas situaciones.

4.1 Función `data_split`

Para poder dividir el conjunto de datos, en primer lugar se importó la función necesaria de Scikit. Después, se diseñó una función que, recibiendo como argumento de entrada el nombre del músculo deseado, dividiera el DataFrame correspondiente en una matriz de datos y su vector de clases. Después, con la función `train_test_split` se crearon los conjuntos de entrenamiento y validación, fijando el tamaño del conjunto de validación, y añadiendo el argumento `random_state` para conseguir reproducibilidad.

```
from sklearn.model_selection import train_test_split
```

```
def data_split(musculo):

  data = pd.read_csv(f'/content/{musculo}', index_col=0)

  X = data.loc[:,['Max', 'Mean', 'Std','Median','RMS', 'SSI']]
  y = data.iloc[:, -1]
```

```
  X_train, X_val, y_train, y_val = train_test_split(X, y, test_size =
4, random_state = 1, stratify = y)

  return X_train, X_val, y_train, y_val
```

4.2 Creación de un modelo inicial, entrenamiento y evaluación

Una vez se dispuso de los conjuntos de entrenamiento y validación, se procedió a la programación del entrenamiento de un modelo inicial, en este caso un árbol de clasificación. En este caso también se introdujo el argumento `random_state` para asegurar reproducibilidad.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

emg_model = DecisionTreeClassifier(random_state = 1)
emg_model.fit(X_train, y_train)
```

Una vez entrenado el modelo, se puede evaluar predicciones sobre el conjunto de validación, obteniendo también una métrica de rendimiento (*accuracy*). Posteriormente, se buscó mejorar estos resultados configurando parámetros del modelo, como la máxima profundidad, tal y como se detalla en el siguiente apartado.

```
y_pred = emg_model.predict(X_val)
acc = accuracy_score(y_val, y_pred)
print(f'Accuracy: {acc}')
```

4.3 Mejorando el modelo

4.3.1 Funciones `get_acc` y `busca_max_depth`

El modelo inicial deja sin fijar parámetros determinantes en la precisión del árbol, como la profundidad máxima (`max_depth`). Se puede programar mediante un bucle la búsqueda de la mejor profundidad entre una lista de valores candidatos (`candidate_max_depth`), para implementarla en un segundo modelo y ver si mejora el rendimiento. Para ello, se emplean dos funciones:

- `get_acc`, que genera un modelo tal y como se ha hecho previamente y calcula su *accuracy* dados una profundidad fija y los conjuntos de validación y entrenamiento.
- `busca_max_depth`, que llama a la función anterior tantas veces como valores candidatos a `max_depth` haya en la lista, calculando la accuracy para cada uno y devolviendo el máximo al final.

```
def get_acc(depth, X_train, X_val, y_train, y_val):
    model = DecisionTreeClassifier(max_depth = int(depth),
random_state = 1)
    model.fit(X_train, y_train)
    preds_val = model.predict(X_val)
    acc = accuracy_score(y_val, preds_val)
    return(acc)

def busca_max_depth(candidate_max_depth, X_train, X_val, y_train,
y_val):
  accs = []
  for max_depth in candidate_max_depth:
      acc = get_acc(max_depth, X_train, X_val, y_train, y_val)
      accs.append(acc)
  best_tree_depth = int(candidate_max_depth[accs.index(max(accs))])

  return best_tree_depth
```

4.3.2 Creación del modelo final: función `entrenar_evaluar_modelo`

Para realizar la evaluación de los modelos en los diferentes músculos, se embebieron los procesos de creación del modelo, entrenamiento y prueba dentro de la función `entrenar_evaluar_modelo`. Esta recibe como parámetros el nombre del músculo a analizar y los hiperparámetros de control de complejidad del árbol (como `max_depth`), devolviendo el modelo ajustado, su métrica de exactitud (*accuracy*) y la matriz de confusión con los resultados sobre el conjunto de validación.

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

def entrenar_evaluar_modelo(musculo, random_state=1):
    # 1. Obtener los datos divididos
    X_train, X_val, y_train, y_val = data_split(musculo)

    # 2. Entrenar el modelo
    max_depth = busca_max_depth(candidate_max_depth, X_train, X_val,
y_train, y_val)
    print(f'La mejor profundidad para el musculo {musculo} es:
{max_depth}')
    modelo = DecisionTreeClassifier(max_depth=max_depth,
```

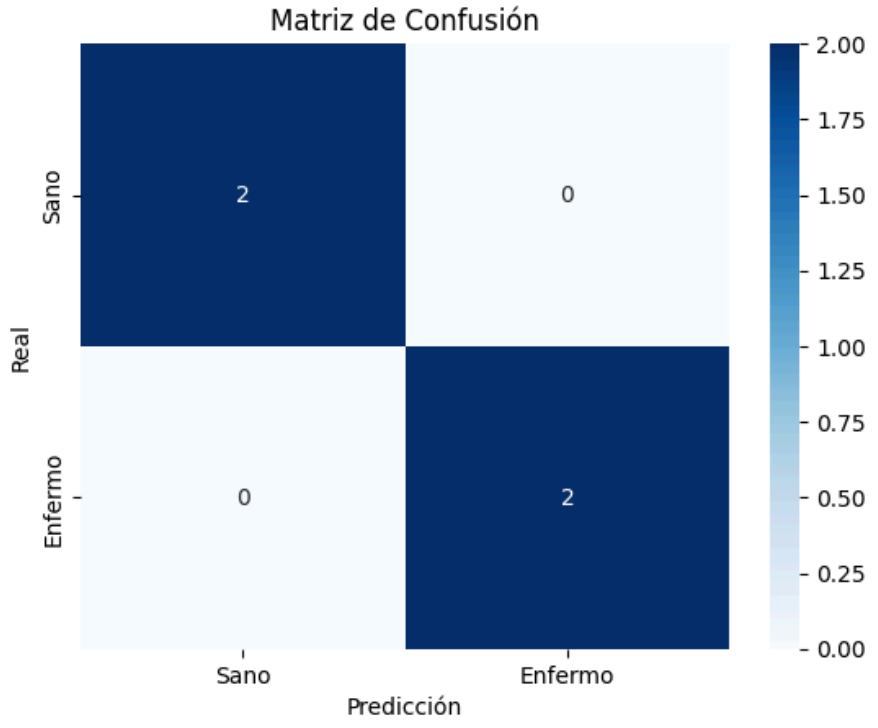
```
random_state=random_state)
    modelo.fit(X_train, y_train)

    # 3. Predicción y métricas
    y_pred = modelo.predict(X_val)
    acc = accuracy_score(y_val, y_pred)
    print(f'La precisión alcanzada para el musculo {musculo} es
{acc}')
    conf_matrix = confusion_matrix(y_val, y_pred)
    sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
xticklabels=['Sano', 'Enfermo'], yticklabels=['Sano', 'Enfermo'])
    plt.xlabel('Predicción')
    plt.ylabel('Real')
    plt.title('Matriz de Confusión({musculo})')
    plt.show()

    return modelo, acc
```

- Ejemplo de salida:

```
La mejor profundidad para el musculo Biceps es: 3
La precisión alcanzada para el musculo Biceps es 1.0
```



5. Resultados

Se ha creado un modelo para cada músculo, obteniendo diferentes resultados en cada caso concreto. Los resultados se recogen en las Figuras 2 y 3, y en la Tabla 1:

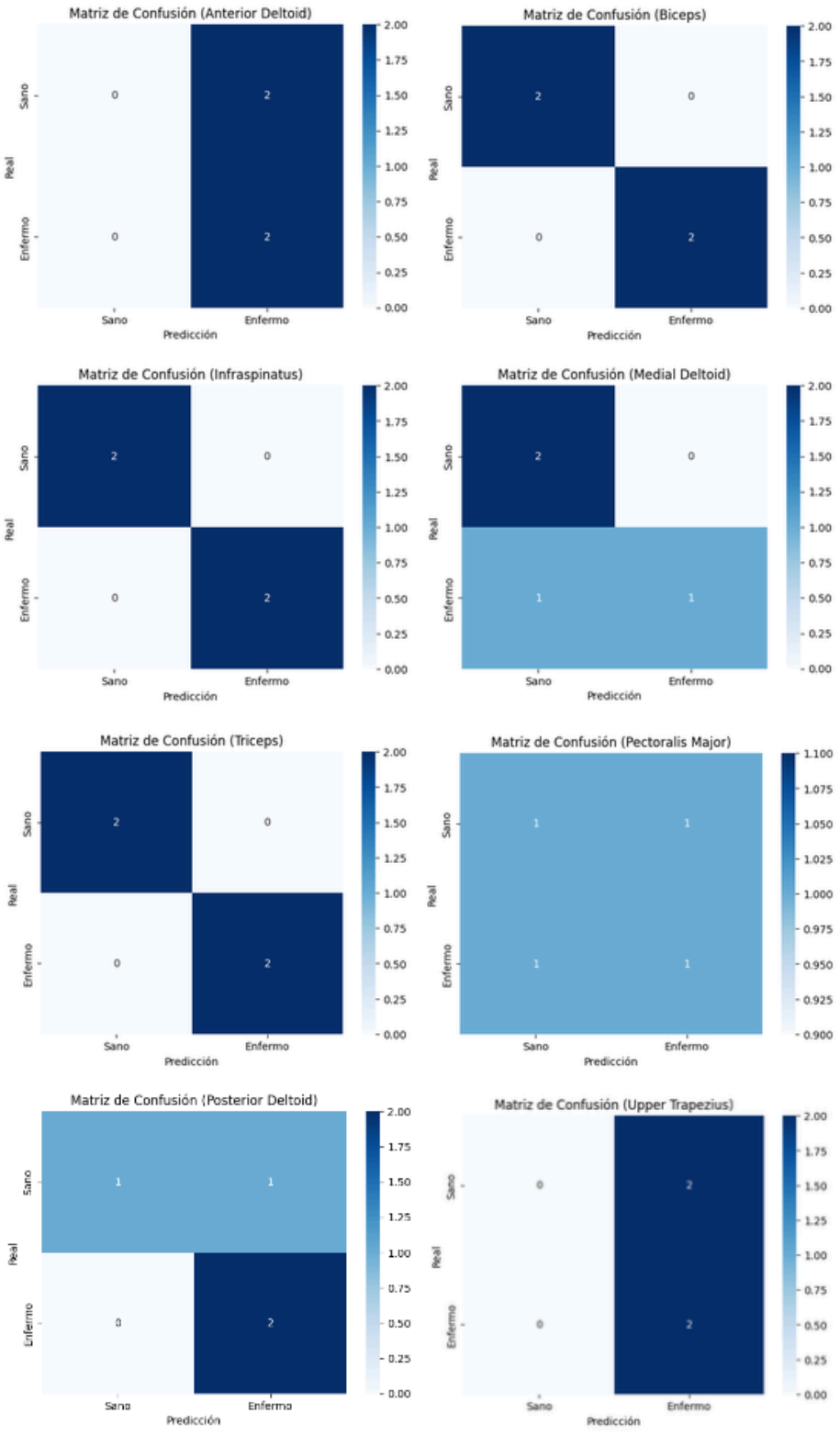


Fig 2. Matrices de confusión para los diferentes músculos.

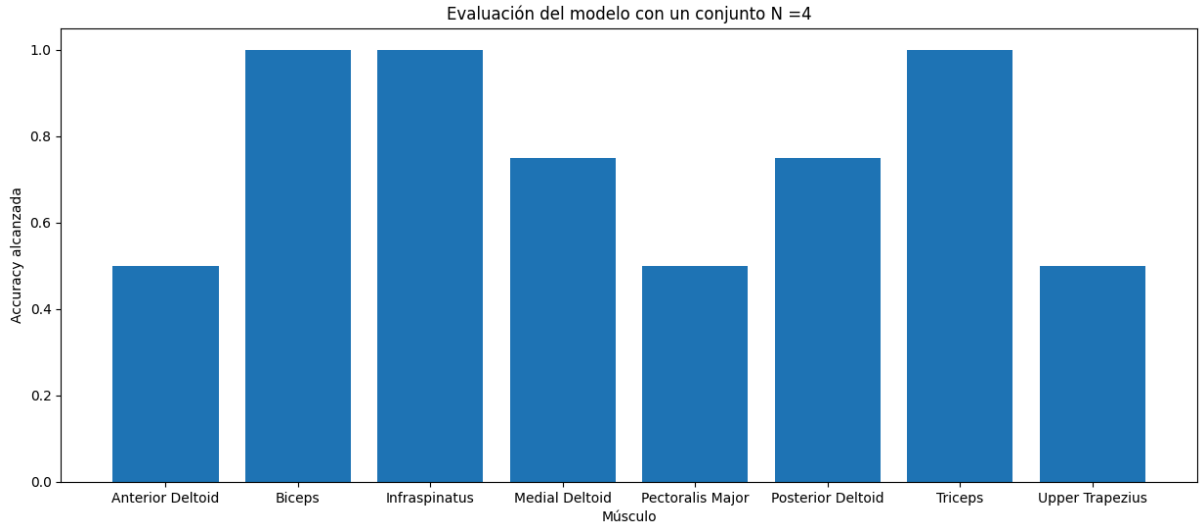


Fig 3. Comparación métrica de exactitud entre modelos.

Músculo	*Accuracy*	*Sensibilidad*	*Especificidad*
Deltoides anterior	0,5	-	0,5
Bíceps	1	1	1
Infraespinoso	1	1	1
Deltoides medio	0.75	0,67	1
Tríceps	1	1	1
Pectoral mayor	0,5	0,5	0,5
Deltoides posterior	0,75	1	0,67
Trapecio superior	0,5	-	0,5
Media:	0,75	0,64	0,77

Tabla 1.Resumen de métricas para los diferentes músculos (figuran con '-' valores que no se han podido calcular porque cuentan con un 0 en el denominador).

Tal y como se puede observar, los modelos generados para los diferentes músculos proporcionan diferentes valores de exactitud, y en vista de las matrices de confusión no se puede sacar una conclusión general sobre la especificidad y sensibilidad del modelo.Podemos destacar que la métrica que alcanza un valor más alto es la especificidad, siendo alta incluso en modelos que no presentaban una exactitud total. Esto indica una baja posibilidad de recibir un resultado negativo siendo una persona afectada, y que por tanto los modelos discriminan bastante bien a las personas sanas. Sin embargo, en algunos casos la sensibilidad o la precisión son poco representativas o mejorables.

6. Conclusión y líneas futuras

Se ha desarrollado de forma completa el proceso de búsqueda y limpieza de los datos, así como el entrenamiento del modelo y su evaluación. Los resultados obtenidos entran dentro de lo esperado, pues los modelos no tienen de media, una precisión totalmente mala (0,77), si bien en algunos casos concretos es mejorable. Por otro lado, en vista de los resultados, la selección de variables y características puede no haber sido óptima. De cara a nuevas versiones del modelo, sería necesario revisar literatura en busca de variables más específicas que describan mejor el problema y cuya combinación consiga elevar la precisión.

Uno de los principales problemas enfrentados ha sido el bajo tamaño de la muestra, impidiendo un buen balance entre sujetos de entrenamiento y validación. Esto no solo afecta a la calidad del entrenamiento, si no también a la representatividad del conjunto de validación. Para esquivar dicha limitación sería necesario implementar mecanismos de *K-Fold Cross Validation* con el objetivo de estimar el rendimiento del modelo en datos no vistos minimizando el sesgo de partición. Otra forma de eliminar el sesgo que aporta la partición de los datos en un conjunto de validación poco representativo al cálculo de las métricas de precisión sería emplear un algoritmo de *bootstrap* para calcular unos intervalos de confianza. Sin embargo, para conseguir entrenar un modelo con mejores resultados es imprescindible ampliar la búsqueda de bases de datos que provean una mayor cantidad de individuos, tanto para entrenamiento como para validación.

Por último, sería interesante, en nuevas versiones, probar otro tipo de algoritmos de Machine Learning, tales como Random Forest o modelos de regresión logística.

7. Referencias

[1] Israely, Sharon (2018), “EMG datasets form hand-reaching movements for multiple directions of healthy and post-stroke individuals”, Mendeley Data, V1, doi: 10.17632/f4hh43nd78.1
[2] IEEE Staff, . (2009). *2009 4th International IEEE/EMBS Conference on Neural Engineering*. I E E E.
[2] *TENCON 2023 - 2023 IEEE Region 10 Conference*. (2023). IEEE.
[3] Phinyomark, A., Phukpattaranont, P., & Limsakul, C. (2012). Feature reduction and selection for EMG signal classification. *Expert Systems with Applications*, *39*(8), 7420–7431. https://doi.org/10.1016/j.eswa.2012.01.102
[4] Khan, A., Waris, A., Ullah, A., & Qammar, S. (2023). Optimizing Feature Reduction and Selection Techniques for Surface Electromyography. *2023 International Conference on Digital Futures and Transformative Technologies, ICoDT2 2023*. https://doi.org/10.1109/ICoDT259378.2023.10325778